

# **LAPORAN**

## **WEB SERVICE CONTAINER: APACHE, NGINX, REVERSE PROXY, DAN TLS**

Disusun untuk memenuhi tugas

Mata Kuliah: Workshop DevOps

Dosen Pengampu : Ferry Astika Saputra



Oleh :

Mohammad Affan Hendi Firmansyah (3126640049)

**KELAS 1**

**SARJANA TERAPAN LANJUT JENJANG**

**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

**2026**

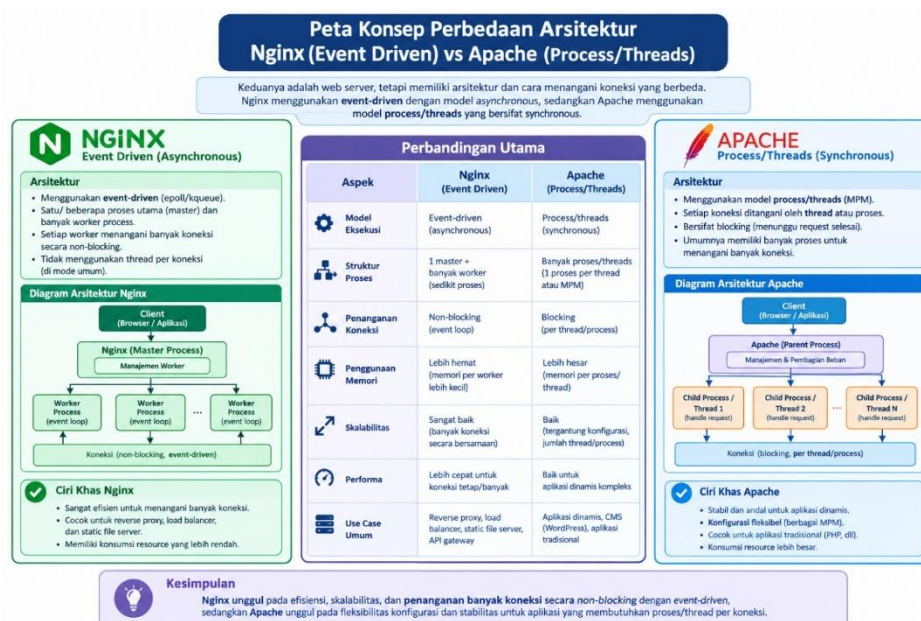
## 1. TUJUAN PRAKTIKUM

- Deployment Web Server Apache & Nginx: Mengoperasikan Apache (httpd:2.4-alpine) dan Nginx (nginx:alpine) dalam kontainer dengan konfigurasi kustom dan bind mount sistem berkas.
- Implementasi Reverse Proxy & Ingress Gateway: Mengonfigurasi Nginx sebagai gerbang tunggal yang memisahkan akses publik dari backend internal (/ke Apache dan /api/ke Flask).
- Simulasi TLS/HTTPS & Redirection 301: Menerapkan sertifikat TLS *self-signed*, menegosiasikan enkripsi modern TLS 1.3 pada port 8443, dan memvalidasi pengalihan otomatis HTTP (port 8080) ke HTTPS (port 8443).
- Isolasi Jaringan & Penerusan Header: Membatasi paparan port (*least exposure*) dengan mengisolasi backend pada jaringan privat (web-net) serta meneruskan header identitas (X-Forwarded-For, X-Forwarded-Proto, Host).
- Analisis Keamanan DevSecOps: Mengevaluasi proteksi *read-only mount* (:ro), risiko private key pada bind mount, serta mitigasi *header spoofing*.

## 2. DASAR TEORI: KOMPARASI APACHE HTTPD DAN NGINX

Dalam arsitektur *containerized web services*, Apache dan Nginx memiliki karakteristik yang saling melengkapi:

| Paramet er Evaluasi   | Apache HTTP Server (httpd)                    | Nginx Reverse Proxy                         | Analisis Peran Arsitektur                                                                |
|-----------------------|-----------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------------------|
| Model Arsitektur      | Process/thread-per-request (MPM Event/Worker) | Event-driven, non-blocking asynchronous     | Nginx unggul menangani ribuan koneksi konkuren; Apache unggul pada kompatibilitas modul. |
| Direktori Konfigurasi | /usr/local/apache2/conf                       | /etc/nginx                                  | Nginx menggunakan blok server& location; Apache berbasis direktif XML-like.              |
| Docume nt Root        | /usr/local/apache2/htdocs                     | /usr/share/nginx/html                       | Apache berperan sebagai <i>origin server</i> statis; Nginx sebagai <i>ingress</i> .      |
| Fitur Reverse Proxy   | Modul mod_proxy                               | Direktif bawaan proxy_pass                  | Nginx dirancang natively sebagai reverse proxy dengan overhead memori minimal.           |
| Peran pada Praktikum  | Penyaji konten web internal (/)               | TLS Ingress & Routing Gateway (/ dan /api/) | Pola kombinasi memisahkan terminasi TLS dari eksekusi backend.                           |



## 3. LANGKAH PRAKTIKUM & BUKTI EKSEKUSI

### a. Arsitektur Multi-Service Compose

Arsitektur dideklarasikan dalam berkas `compose.yaml` yang menghubungkan layanan proxy, apache-web, dan flask-appke dalam jaringan bridge web-net. Hanya kontainer proxy yang memublikasikan port ke host (8080 dan 8443):

```

GNU nano 8.7.1                                     compo
services:
  proxy:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/conf:/etc/nginx/conf.d:ro
      - ./certs:/etc/nginx/certs:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - web-net
    depends_on:
      apache-web:
        condition: service_started
      flask-app:
        condition: service_healthy

  apache-web:
    image: httpd:2.4-alpine
    volumes:
      - ./apache/sites:/usr/local/apache2/htdocs:ro
    networks:
      - web-net

```

## b. Status Operasional Layanan dan Batas Paparan Port

Stack dijalankan dengan docker compose up -d --build. Status operasional kontainer diperiksa melalui docker

```
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ docker compose ps
```

| NAME               | PORTS    | IMAGE            | COMMAND                       | SERVICE    | CREATED            | STATUS                 |
|--------------------|----------|------------------|-------------------------------|------------|--------------------|------------------------|
| bab-4-apache-web-1 | 80/tcp   | httpd:2.4-alpine | "httpd-foreground"            | apache-web | About a minute ago | Up About a minute      |
| bab-4-flask-app-1  | 5000/tcp | bab-4-flask-app  | "unicorn --bind 0.0.0.0:5000" | flask-app  | 22 seconds ago     | Up 21 seconds (health) |

compose ps:

Gambar 1: Output docker compose ps membuktikan ketiga kontainer berstatus Up (flask-app berstatus healthy), dengan port 8080 dan 8443 hanya dipublikasikan oleh proxy.

## c. Cuplikan Log Terpadu Sistem

Log gabungan dari ketiga kontainer diperiksa via docker compose logs untuk memvalidasi inisialisasi worker process dan proteksi sistem berkas:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ docker compose logs --tail 100
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.2.
Set the 'ServerName' directive globally to suppress this message
apache-web-1 | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.2.
Set the 'ServerName' directive globally to suppress this message
apache-web-1 | [Tue Sep 22 01:50:58.2065 2026] [mpm_event notice] [pid 1:tid 1] AH00489: Apache/2.4.68 (Unix) configu
red -- running normal operations
apache-web-1 | [Tue Sep 22 01:50:58.2065 2026] [core:notice] [pid 1:tid 1] AH00694: Command Line: 'httpd -D FOREGROUND'
flask-app-1 | [2026-09-22 01:50:58.35 +0800] [1] [INFO] Starting unicorn 23.0.0
flask-app-1 | [2026-09-22 01:50:58.35 +0800] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
flask-app-1 | [2026-09-22 01:50:58.35 +0800] [1] [INFO] Using worker: sync
flask-app-1 | [2026-09-22 01:50:58.35 +0800] [7] [INFO] Booting worker with pid: 7
flask-app-1 | [2026-09-22 01:50:58.35 +0800] [9] [INFO] Booting worker with pid: 9
proxy-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file sys
tem?)
proxy-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy-1 | /docker-entrypoint.sh: Configuration complete; ready for start up

```

Gambar 2: Log gabungan membuktikan Apache MPM event aktif, Unicorn menjalankan 2 worker process, dan skrip Nginx mendeteksi read-only filesystem pada default.conf.

## d. Pengujian Pengalihan HTTP 301 ke HTTPS

Akses HTTP biasa pada port 8080 diuji dengan curl -I http://localhost:8080:

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ curl -I http://localhost:8080/
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.5
Date: Tue, 22 Sep 2026 01:51:23 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://localhost:8443/

```

Gambar 3: Respons HTTP/1.1 301 Moved Permanently dengan Location: https://localhost:8443/ membuktikan aturan redirect Nginx bekerja sempurna.

## e. Pengujian Akses HTTPS Apache via Nginx Reverse Proxy

Pengujian jalur root / dilakukan melalui HTTPS terenkripsi (curl -k -i https://localhost:8443/):

```

affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ curl -k -i https://localhost:8443/
HTTP/1.1 200 OK
Server: nginx/1.31.5
Date: Tue, 22 Sep 2026 01:51:27 GMT
Content-Type: text/html
Content-Length: 282
Connection: keep-alive
Last-Modified: Tue, 22 Sep 2026 01:36:24 GMT
ETag: "11a-66c8650a36ad"
Accept-Ranges: bytes
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referer-Policy: no-referrer

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>DevSecOps Bab 4</title>
</head>
<body>
  <div>
    <h1>Apache di Belakang Nginx</h1>
    <p>Lab 4 berhasil dilalui melalui reverse proxy.</p>
    <p><a href="/api/*">Flask API</a></p>
  </div>
</html>

```

Gambar 4: Respons HTTP/1.1 200 OK menyajikan konten HTML Apache dengan header keamanan X-Frame-Options, X-Content-Type-Options, dan CSP.

#### f. Pengujian Akses Endpoint API Microservice Flask

Pengujian perutean API dilakukan ke endpoint /api(curl -k https://localhost:8443/api):

```
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ curl -k https://localhost:8443/api/
{"forwarded_proto":"https","message":"API Bab 4 berhasil diakses melalui Nginx","service":"flask-app","status":"ok"}
```

Gambar 5: Respons JSON dari Flask mengonfirmasi penerusan header X-Forwarded-Proto bernilai https dan client\_ip teridentifikasi.

#### g. Validasi Kriptografi Handshake TLS 1.3 via OpenSSL

Audit cipher suite dan sertifikat dilakukan melalui openssl s\_client -connect localhost:8443:

```
affan@PC-PDEAFFAN-PDEK348:~/docker-lab/bab-4$ openssl s_client \
-connect localhost:8443 \
-servername localhost \
-brief </dev/null
Connecting to 127.0.0.1
depth=0 CN=localhost, O=DevSecOps Docker Lab
verify error:num=18:self-signed certificate
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN=localhost, O=DevSecOps Docker Lab
Hash used: SHA256
Signature type: rsa_pss_rsae_sha256
Verification error: self-signed certificate
Negotiated TLS1.3 group: X25519MLKEM768
DONE
```

Gambar 6: Sesi TLS menegosiasikan protokol TLSv1.3, cipher TLS\_AES\_256\_GCM\_SHA384, dan algoritma pertukaran kunci hybrid X25519MLKEM768.

### 4. ANALISIS MASALAH DAN DIAGNOSTIK (WAJIB)

Selama pelaksanaan praktikum, ditemukan tiga kondisi diagnostik penting yang dianalisis:

#### a. Peringatan FQDN Apache (AH00558):

- *Temuan:* Pada log Apache (Gambar 2), muncul pesan AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.21.0.2.
- *Diagnosis:* Direktif ServerName belum didefinisikan secara eksplisit di berkas httpd.conf, sehingga daemon Apache otomatis menggunakan alamat IP kontainer sebagai fallback. Masalah ini bersifat non-fatal, namun pada lingkungan produksi harus ditetapkan direktif ServerName localhost:80 untuk mencegah perilaku redirection yang anomali.

#### b. Peringatan Modifikasi Sistem Berkas Nginx:

- *Temuan:* Skrip inisialisasi Nginx mencatat can not modify /etc/nginx/conf.d/default.conf (read-only file system?).
- *Diagnosis:* Skrip bawaan 10-listen-on-ipv6-by-default.sh mencoba menyuntikkan listen IPv6 ke konfigurasi default. Karena volume dipasang dengan atribut :ro (read-only), penulisan ditolak oleh kernel. Hal ini justru memvalidasi bahwa mekanisme isolasi berkas berhasil melindungi konfigurasi host dari modifikasi kontainer.

#### c. Peringatan Verifikasi Sertifikat Self-Signed:

- *Temuan:* Perintah curl mewajibkan flag -k (insecure) dan OpenSSL mencatat verify error:num=18:self-signed certificate.
- *Diagnosis:* Sertifikat dibuat secara mandiri (self-signed) tanpa penandatanganan oleh Certificate Authority (CA) terpercaya yang ada pada trust store lokal sistem. Pada pengujian lab hal ini wajar, namun di produksi wajib menggunakan sertifikat resmi.

### 5. ANALISIS RISIKO KEAMANAN & OPERASIONAL (DEVSECOPS) (WAJIB)

- Risiko Private Key pada Bind Mount: Berkas kunci privat TLS (lab.key) dipetakan via bind mount biasa. Jika host atau kontainer memiliki kelemahan perizinan berkas (file permission lax), kunci privat dapat diekstraksi penyerang untuk mendekripsi rekaman lalu lintas jaringan (eavesdropping). Kunci privat seharusnya dikelola melalui Docker Secrets atau Secret Manager terenkripsi.
- Risiko Header Spoofing & Trust Boundary: Microservice Flask mengandalkan header X-Forwarded-Proto untuk mendeteksi skema koneksi aman. Jika kontainer flask-app tidak diisolasi dalam jaringan privat dan portnya dibuka ke host, penyerang dapat memotong proxy (bypass) dan menyuntikkan header palsu untuk mengelabui logika otorisasi aplikasi.
- Penerapan Header Keamanan HTTP: Nginx dikonfigurasi menyuntikkan X-Frame-Options: DENY (mitigasi serangan Clickjacking), X-Content-Type-Options: nosniff (mitigasi MIME-confusion attack), dan Content-Security-Policy: "default-src 'self'" (mitigasi XSS).

- Analisis Kriptografi TLS 1.3 & Post-Quantum Key Exchange: Sesi HTTPS menegosiasikan TLS\_AES\_256\_GCM\_SHA384 dengan algoritma pertukaran kunci hybrid X25519MLKEM768. Kombinasi kurva eliptik X25519 dan kisi matematis ML-KEM-768 memberikan ketahanan terhadap ancaman komputasi kuantum di masa depan (*quantum-resistant cryptography*).

## 6. REKOMENDASI PERBAIKAN UNTUK LINGKUNGAN PRODUKSI (WAJIB)

- **Sertifikat CA Publik Terpercaya & ACME:** Ganti sertifikat *self-signed* dengan sertifikat dari CA publik (Let's Encrypt / DigiCert) menggunakan protokol ACME (*Certbot*) untuk rotasi sertifikat otomatis sebelum masa berlaku 90 hari berakhir.
- **Penyimpanan Kunci via Docker Secrets / Vault:** Hilangkan bind mount direktori certs dan gunakan mekanisme *Docker Secrets* (`/run/secrets/tls_key`) dengan permission 0400 atau integrasi HashiCorp Vault.
- **Enkripsi End-to-End (mTLS):** Untuk kepatuhan standar industri (PCI-DSS / HIPAA), terapkan TLS timbal balik (*mutual TLS*) antara Nginx reverse proxy dan backend Flask guna mencegah penyadapan lalu lintas pada jaringan internal host.
- **Pengguna Non-Root & Pembatasan Sumber Daya:** Pastikan seluruh kontainer (khususnya Flask dan Nginx) dijalankan dengan pengguna non-root (UID 10001) serta dibatasi alokasi CPU dan memorinya via blok `deploy.resources.limits`.

## 7. EVALUASI DAN LATIHAN MANDIRI

- **Mengapa reverse proxy tidak seharusnya menjalankan semua logik aplikasi?**  
*Jawaban:* Reverse proxy dirancang untuk menangani tugas tingkat transport dan protokol (seperti terminasi TLS, perutean jalur URL, kompresi respons, caching aset statis, dan mitigasi DDoS). Menjalankan logika bisnis kompleks di reverse proxy akan membebani siklus event-loop non-blocking, meningkatkan latensi koneksi, mengaburkan batas tanggung jawab arsitektural (*separation of concerns*), dan memperluas *attack surface* pada lapisan terluar jaringan.
- **Apa perbedaan TLS termination dan end-to-end TLS?**  
*Jawaban:* Pada *TLS termination*, koneksi terenkripsi HTTPS didekripsi di reverse proxy, lalu trafik diteruskan ke backend dalam bentuk HTTP teks biasa (unencrypted) melalui jaringan privat. Pada *end-to-end TLS*, enkripsi dipertahankan dari klien hingga ke backend, di mana proxy melakukan re-enkripsi atau meneruskan koneksi TLS (SNI pass-through / mTLS) sehingga lalu lintas pada jaringan internal tetap terlindungi dari penyadapan internal.
- **Bagaimana cara mengisolasi backend agar tidak langsung diakses dari host?**  
*Jawaban:* Cara terbaik adalah: (1) Menempatkan kontainer backend pada user-defined bridge network internal (web-net) tanpa mendeklarasikan pemetaan port (ports) ke host; (2) Hanya mengekspos port melalui kontainer reverse proxy; dan (3) Jika port backend terpaksa dibuka ke host untuk kebutuhan diagnostik, batasi binding secara ketat pada alamat loopback lokal host (127.0.0.1:5000:5000), bukan di 0.0.0.0.
- **Apa konsekuensi menyimpan private key TLS di bind mount?**  
*Jawaban:* Kunci privat yang disimpan pada direktori bind mount rentan terbaca oleh pengguna host non-root jika izin berkas terlalu longgar (*lax permissions*). Selain itu, berkas kunci berisiko terkomit secara tidak sengaja ke repositori Git publik (*credential leak*), serta tidak terlindungi oleh mekanisme enkripsi saat istirahat (*encryption at rest*) bawaan sistem container engine.
- **Bandingkan log Nginx dan log Apache dari sisi format dan kegunaan debugging.**  
*Jawaban:*  
Log Apache: Menggunakan format Common Log Format (CLF) atau Combined. Error log Apache berorientasi modul (`[module:level] [pid:tid] AHxxxx: message`), sangat informatif untuk mendiagnosis kegagalan internal modul web server (seperti MPM event, modul rewriting, atau izin akses berkas direktori `htdocs`). Log Nginx: Menyediakan variabel upstream yang kaya (`$upstream_addr`, `$upstream_status`, `$upstream_response_time`). Nginx access dan error log sangat unggul untuk menelusuri performa jaringan, latensi layanan backend, kesalahan resolusi DNS internal kontainer, serta pemecahan masalah koneksi reverse proxy (*connection refused* atau *502 Bad Gateway*).

## 8. KESIMPULAN

Praktikum Bab 4 membuktikan keberhasilan implementasi arsitektur web service multi-kontainer modern dengan memadukan Apache sebagai penyaji konten statis, Flask sebagai microservice API, dan Nginx sebagai reverse proxy sekaligus gerbang TLS ingress. Pengujian cURL dan OpenSSL membuktikan berfungsinya pengalihan otomatis HTTP 301 ke HTTPS, perutean path `/` dan `/api`, penerusan header identitas (X-Forwarded-Proto), serta negosiasi enkripsi modern TLS 1.3 dengan cipher TLS\_AES\_256\_GCM\_SHA384 dan pertukaran kunci pasca-kuantum X25519MLKEM768. Dari perspektif DevSecOps, isolasi backend tanpa paparan port host, penegakan header keamanan HTTP, penggunaan *read-only bind mount* (`:ro`), serta orkestrasi dependensi cerdas (*service healthy*) berhasil membentuk perimeter pertahanan yang andal sesuai prinsip *Least Privilege* dan *Least Exposure*.